Practical 2: Source Code and Output

Code:

```
# ============================================================
# MODEL-BASED INTELLIGENT AGENT FOR GRID NAVIGATION
# ============================================================

# Import deque for Breadth First Search (BFS).
from collections import deque
# Create the grid.
# 0 = Free cell
# X = Obstacle
grid = [
    ['S',0,0,'X'],
    [0,'X',0,0],
    [0,0,0,'X'],
    ['X',0,0,'G']
]
# Number of rows.
rows = len(grid)
# Number of columns.
cols = len(grid[0])
# Start position.
start = (0,0)
# Goal position.
goal = (3,3)
# Queue stores current node and path.
queue = deque([(start,[start])])
# Internal state (memory): visited cells.
visited = set()
visited.add(start)
# Possible movements.
directions=[(-1,0),(1,0),(0,-1),(0,1)]
print("Grid Navigation Begins\n")
# Variable to store final path.
final_path=None
```

```
# Continue until queue becomes empty.
while queue:
    # Remove first element.
    current,path = queue.popleft()
    # Display current state.
    print("Current Position:",current)
    print("Visited:",visited)
    # Goal test.
    if current==goal:
        final_path=path
        print("\nGoal Reached!")
        break
    # Explore neighbours.
    for dr,dc in directions:
        # Compute neighbour coordinates.
        nr=current[0]+dr
        nc=current[1]+dc
        # Check grid boundary.
        if 0<=nr<rows and 0<=nc<cols:
            # Skip obstacles.
            if grid[nr][nc]=='X':
                continue
            # Skip visited cells.
            if (nr,nc) in visited:
                continue
            # Update internal state.
            visited.add((nr,nc))
            # Add neighbour to queue.
            queue.append(((nr,nc),path+[(nr,nc)]))
print("\n==============================")
if final_path:
    print("Path Found:")
    print(final_path)
else:
print("No path exists.")
```

```
print("\nRational Behaviour Analysis")
print("1. The agent remembers visited cells.")
print("2. It never revisits explored locations.")
print("3. It avoids obstacles.")
print("4. It systematically searches for the goal.")
print("Hence, it behaves as a Model-Based Intelligent Agent.")
```

Output:

Grid Navigation Begins

Current Position: (0, 0)

Visited: {(0, 0)}

Current Position: (1, 0)

Visited: {(1, 0), (0, 1), (0, 0)}

Current Position: (0, 1)

Visited: {(1, 0), (0, 1), (2, 0), (0, 0)}

Current Position: (2, 0)

Visited: {(0, 1), (0, 0), (2, 0), (0, 2), (1, 0)}

Current Position: (0, 2)

Visited: {(0, 1), (2, 1), (0, 0), (2, 0), (0, 2), (1, 0)}

Current Position: (2, 1)

Visited: {(0, 1), (1, 2), (2, 1), (0, 0), (2, 0), (0, 2), (1, 0)}

Current Position: (1, 2)

Visited: {(0, 1), (1, 2), (2, 1), (0, 0), (3, 1), (2, 0), (0, 2), (2, 2), (1, 0)}

Current Position: (3, 1)

Visited: {(0, 1), (1, 2), (2, 1), (0, 0), (3, 1), (2, 0), (0, 2), (2, 2), (1, 0), (1, 3)}

Current Position: (2, 2)

Visited: {(0, 1), (1, 2), (2, 1), (0, 0), (3, 1), (2, 0), (0, 2), (2, 2), (1, 0), (3, 2), (1, 3)}

Current Position: (1, 3)

Visited: {(0, 1), (1, 2), (2, 1), (0, 0), (3, 1), (2, 0), (0, 2), (2, 2), (1, 0), (3, 2), (1, 3)}

Current Position: (3, 2)

Visited: {(0, 1), (1, 2), (2, 1), (0, 0), (3, 1), (2, 0), (0, 2), (2, 2), (1, 0), (3, 2), (1, 3)}

Current Position: (3, 3)

Visited: {(0, 1), (1, 2), (2, 1), (0, 0), (3, 1), (2, 0), (0, 2), (3, 3), (2, 2), (1, 0), (3, 2), (1, 3)}

Goal Reached!

==========

Path Found:

[(0, 0), (1, 0), (2, 0), (2, 1), (3, 1), (3, 2), (3, 3)]

Rational Behaviour Analysis

1. The agent remembers visited cells.

2. It never revisits explored locations.

3. It avoids obstacles.

4. It systematically searches for the goal.

Hence, it behaves as a Model-Based Intelligent Agent.